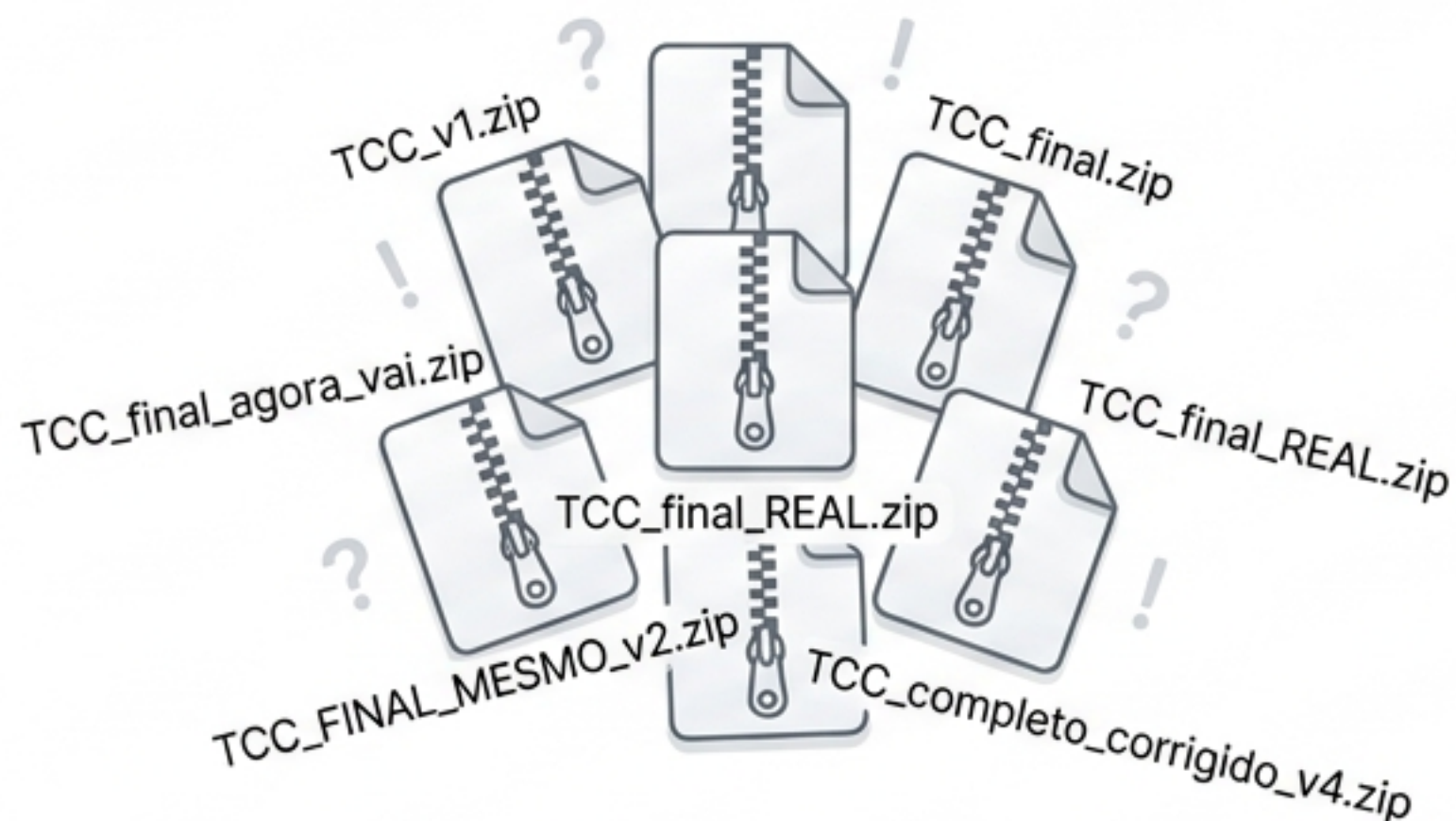


Dominando Git & GitHub

O guia prático de versionamento para transformar
estudantes em desenvolvedores profissionais.

O fim do "TCC_final_agora_vai_v3.zip"

O Caos



Sem versionamento: Arquivos por e-mail ou pendrive. Risco constante de um colega apagar o código do outro.

A Solução



Com **Git & GitHub**: Trabalho simultâneo e seguro. Alterações **unificadas automaticamente** e poder de '**voltar no tempo**' se o aplicativo quebrar.

Ferramentas distintas que trabalham juntas



Git (O Motor Local)

Definição:

Sistema de Controle de Versão.

Como atua:

Roda localmente no seu computador.

Superpoder:

Registra cada alteração, permitindo voltar a versões anteriores se algo der errado.



GitHub (O Hub na Nuvem)

Definição:

Plataforma de Hospedagem em Nuvem.

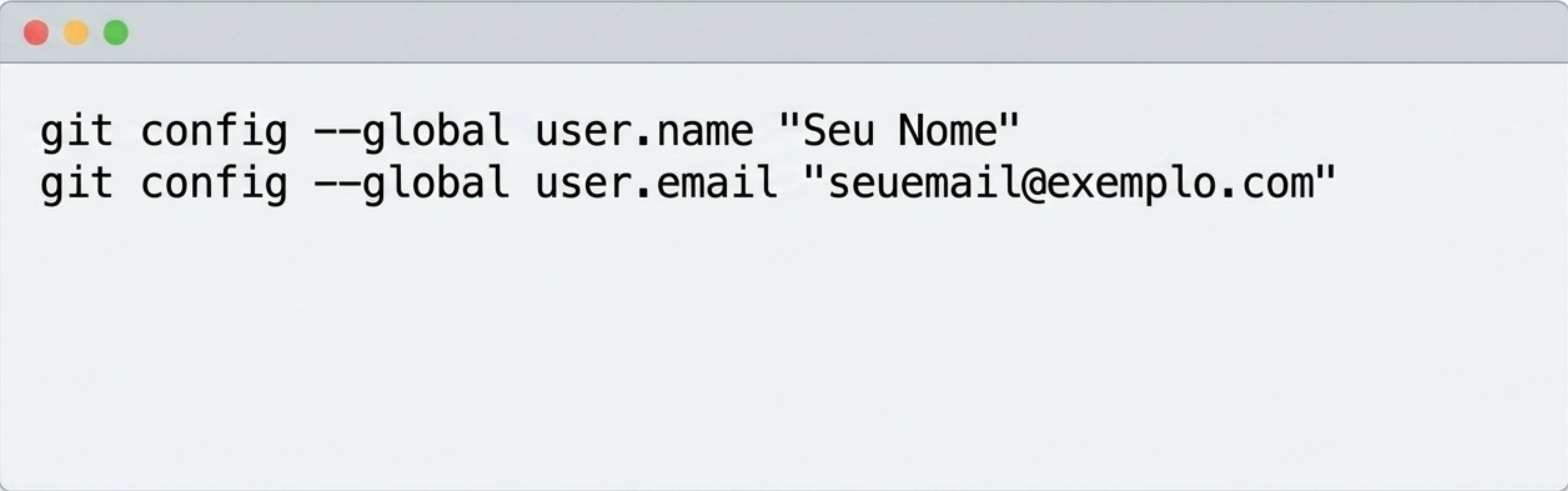
Como atua:

Armazena seus repositórios online.

Superpoder:

Facilita o compartilhamento, o trabalho em equipe e serve como seu portfólio profissional.

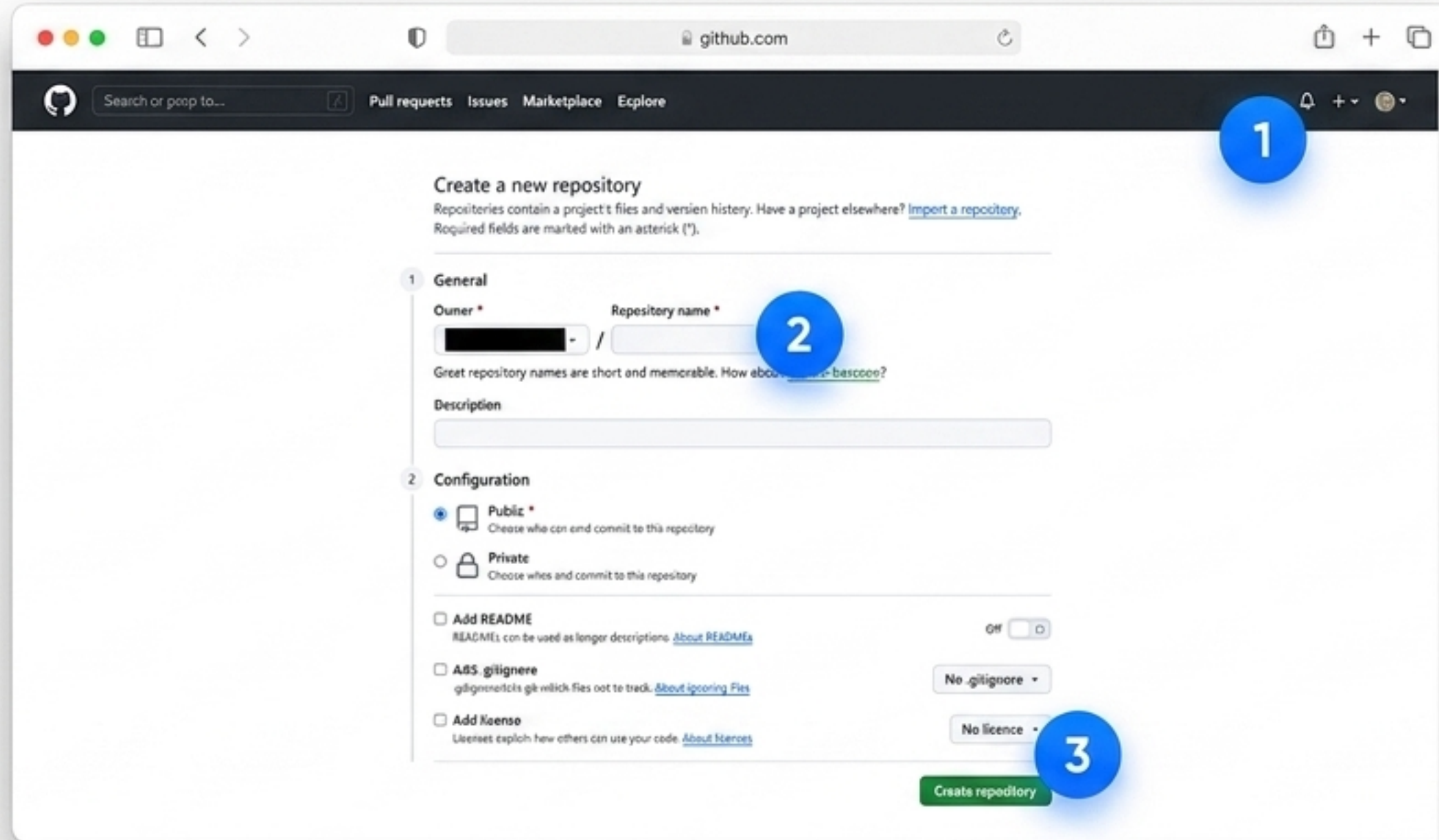
Fase 1: Sua identidade como desenvolvedor



```
git config --global user.name "Seu Nome"  
git config --global user.email "seuemail@exemplo.com"
```

Estes comandos são executados apenas uma vez no seu computador. O objetivo? Garantir que o GitHub saiba exatamente quem é o autor de cada linha de código enviada.

Fase 2: Criando sua base na nuvem

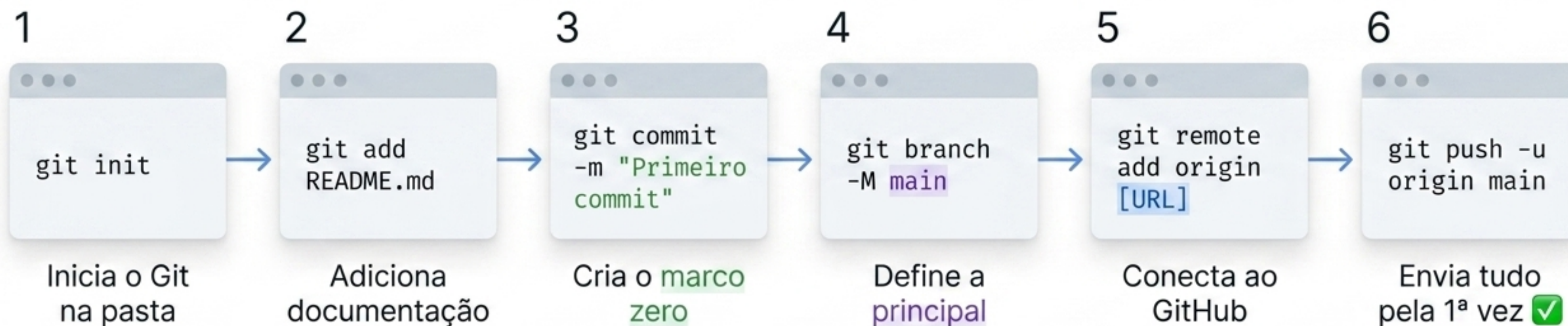


The screenshot shows the GitHub 'Create a new repository' page. The browser address bar shows 'github.com'. The page has a dark header with the GitHub logo, a search bar, and links for 'Pull requests', 'Issues', 'Marketplace', and 'Explore'. A blue circle with the number '1' is placed over the '+ New repository' button in the top right corner. The main content area is titled 'Create a new repository' and includes a sub-header 'Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository.](#) Required fields are marked with an asterisk (*)'. Below this, there are two sections: '1 General' and '2 Configuration'. In the 'General' section, the 'Owner' field is filled with a black box, and the 'Repository name' field is empty, with a blue circle containing the number '2' next to it. The 'Description' field is also empty. In the 'Configuration' section, the 'Public' radio button is selected. Below this, there are three checkboxes: 'Add README' (unchecked), 'Add .gitignore' (unchecked), and 'Add license' (unchecked). To the right of these checkboxes are dropdown menus for '.gitignore' (set to 'No .gitignore') and 'license' (set to 'No license'). A green 'Create repository' button is at the bottom right, with a blue circle containing the number '3' next to it.

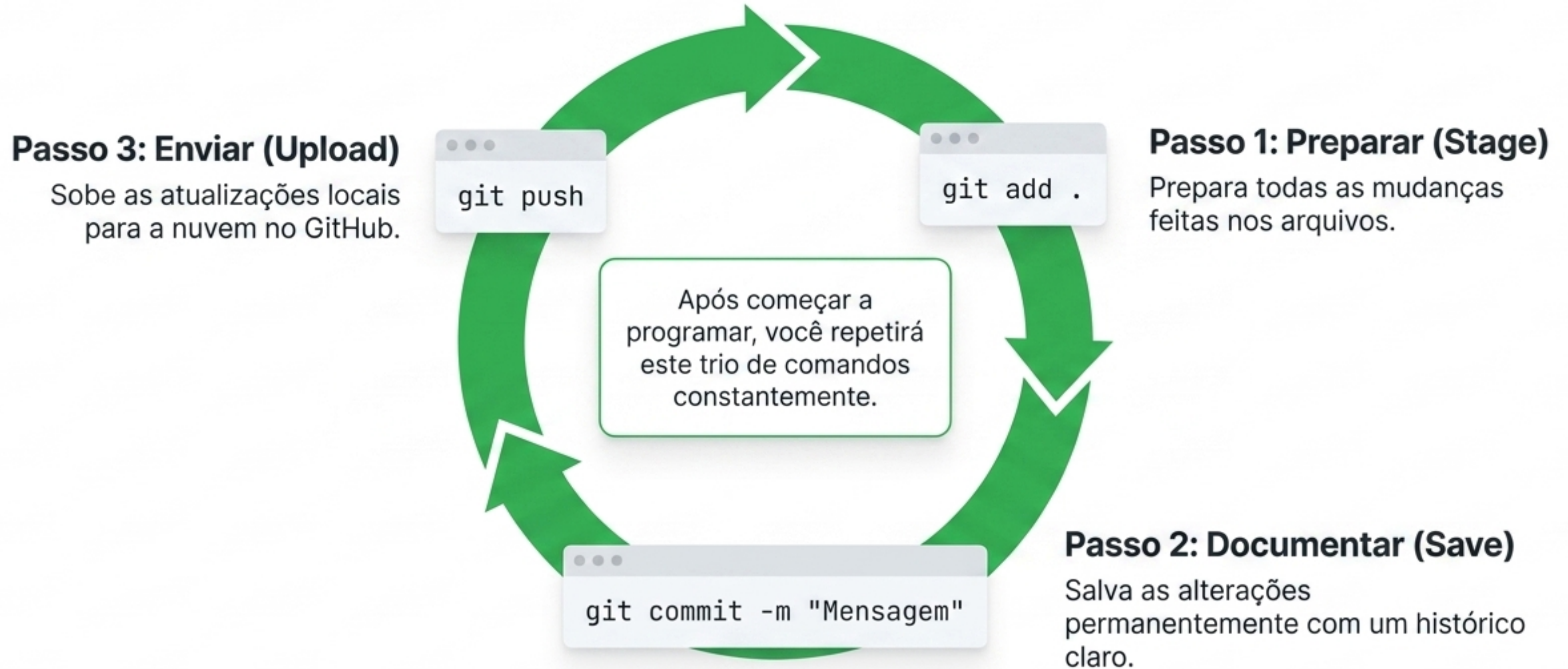
- 1 Acessar **github.com** e clicar em **"New repository"**.
- 2 Inserir o nome do repositório (ex: **MeuProjeto**, **AppEscola**, **CalculadoraJS**).
- 3 Clicar no botão verde **"Create repository"**.

Atenção: Substitua "MeuProjeto" pelo nome real do sistema que você está desenvolvendo.

Fase 3: A primeira conexão (Local → Nuvem)

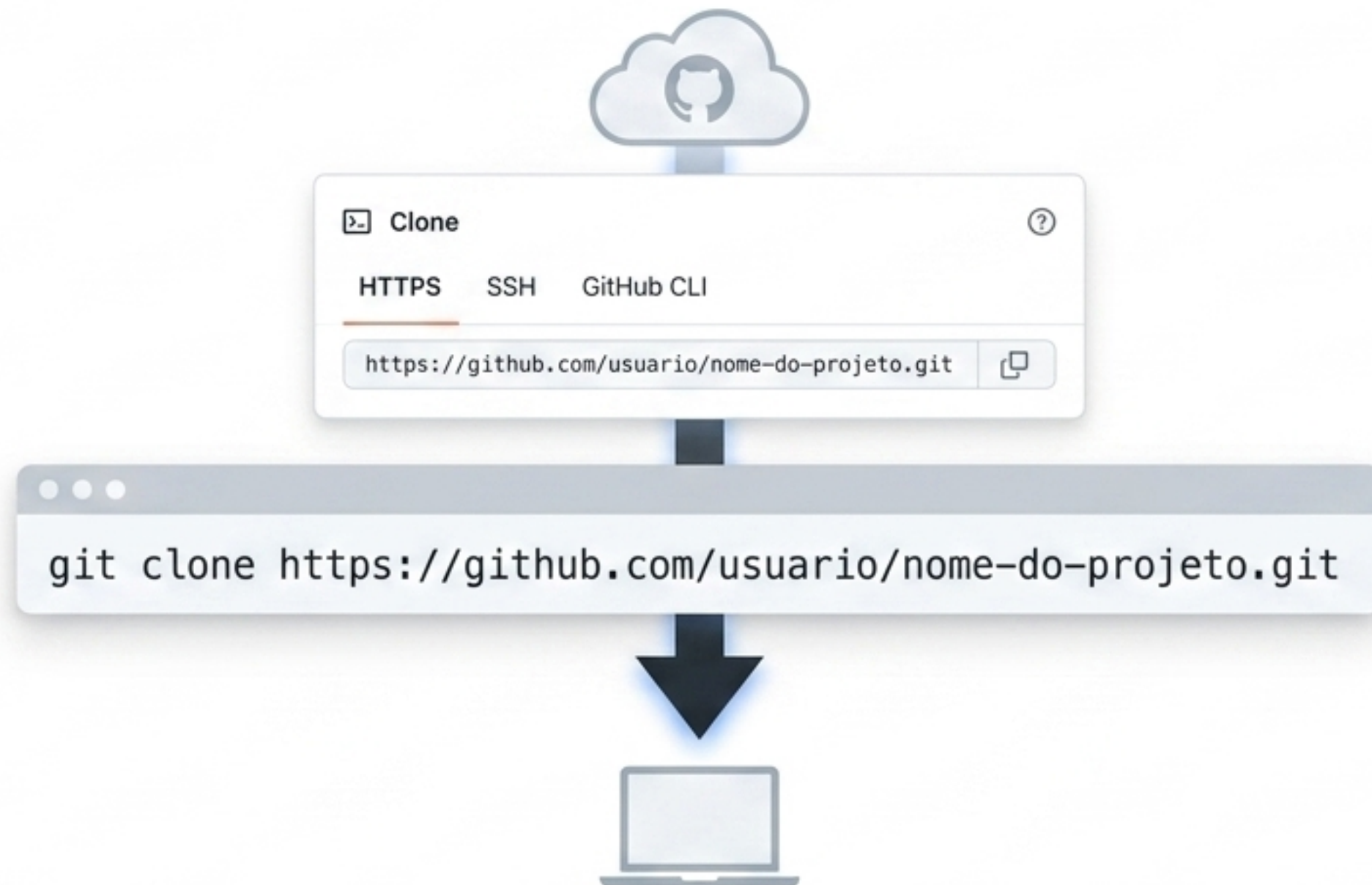


O Ciclo do Dia a Dia



Entrando em um projeto existente

Muitas vezes você não começará do zero, mas ajudará no projeto de um colega.



O Git cria uma cópia exata do repositório remoto no seu computador, baixa todos os arquivos e já configura a conexão com o servidor automaticamente.

Trabalhando com Branchs (Ramificações)



Imagine que o seu projeto é o tronco de uma árvore. Uma branch é um galho criado para testar uma nova funcionalidade (ex: tela de contato) sem o risco de estragar o código principal que já está funcionando.

O fluxo seguro de desenvolvimento

1. Base Estável: Você está na **main**.

Você está na main, o tronco principal do seu projeto, pronto para iniciar uma nova tarefa.

2. Isolar o Trabalho: Cria um novo galho e muda para ele imediatamente.

```
git checkout -b tela-contato
```

3. Desenvolver: Faz seu trabalho no código.

```
git add . e git commit
```

4. Retornar à Base: Volta para a ramificação principal.

```
git checkout main
```

5. Unir as Peças: Traz as melhorias do galho para o tronco principal.

```
git merge tela-contato
```

Regras de ouro para colaboração em equipe

1

Clonagem Inicial

Se for trabalhar no repositório de um colega, comece sempre usando `git clone`.

2

Proteja a Main

Nunca trabalhe diretamente na ramificação principal. Sempre crie uma branch específica para sua tarefa (`git checkout -b nome-da-tarefa`).

3

Integração Segura

Apenas faça o merge para a main depois de finalizar e testar completamente o código na sua branch.

Referência Rápida: Seus Melhores Amigos

Status & Setup

```
git init
```

| **Cria** novo repositório.

```
git status
```

| **Mostra** arquivos alterados.

O Ciclo Diário

```
git add .
```

| **Prepara** arquivos.

```
git commit -m
```

| **Salva** com mensagem.

```
git push
```

| **Envia** para nuvem.

```
git pull
```

| **Baixa** novidades da nuvem.

Ramificações (Branches)

```
git branch
```

| **Lista** ramificações.

```
git checkout
```

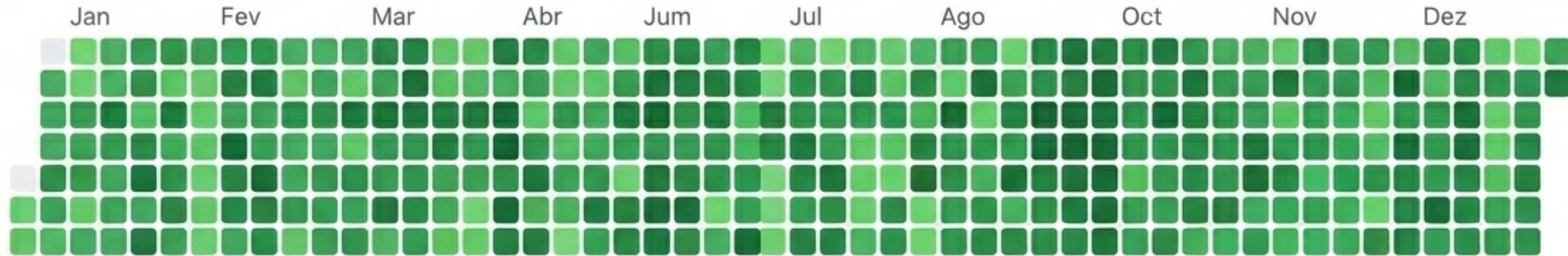
| **Alterna** entre branches.

```
git merge
```

| **Une** alterações.

Lista completa em: comandogit.github.io/

Construindo seu Portfólio Profissional



Aplique este fluxo de versionamento em todos os seus projetos técnicos. Armazenar seu código no GitHub constrói seu portfólio profissional – um diferencial enorme para conquistar seu primeiro estágio como desenvolvedor.

A Dica de Ouro: Nunca passe um dia sem dar um push. Seu perfil ficará cheio de “quadrinhos verdes”, provando sua consistência técnica e dedicação aos estudos.

Para ir além do básico



Documentação Prática

Guia Prático do Git:

rogerdudler.github.io/git-guide

GitHub Docs (PT-BR):

docs.github.com/pt

Atlassian Workflow:

atlassian.com/br/git/tutorials



Cursos Completos (YouTube)

Curso em Vídeo: Git e GitHub para iniciantes absolutos.

Rocketseat: Foco no uso real do mercado de trabalho.



Dicas Rápidas (YouTube)

Filipe Deschamps: Truques e comandos que facilitam o dia a dia do programador.